System Specification: Project CRYSTAL-HYBRID

Autonomous Hybrid RL-LLM Handoff Architecture for *Pokémon Crystal* (GBC)

This specification defines a modular, spec-driven blueprint designed for autonomous implementation via CLI coding agents (e.g., Claude Code, Cursor, or phase-gated developer harnesses). Every component has strict type interfaces, deterministic state invariants, and self-verifying unit/simulation tests so an agentic coding tool can build, test, and debug one phase at a time without polluting its context window.

1. Repository Structure & Agentic Coding Workflow

To prevent an agentic coding tool from hallucinating cross-module imports or breaking emulator timing invariants, the repository is partitioned into isolated, contract-bound subsystems with a compiled .sym lookup table serving as the single source of truth for memory addresses.

Plaintext
```
crystal-hybrid/
├── CLAUDE.md                    # Root instructions, invariants & verification commands for coding agent
├── Makefile                     # Build pokecrystal ROM + .sym, run pytest suites, launch vector sims
├── pyproject.toml               # Python 3.11+, PyBoy>=2.4.0, PyTorch>=2.5, PuffeRL/CleanRL, FastAPI, Pydantic
├── roms/
│   ├── pokecrystal.gbc          # Compiled from pret/pokecrystal (v1.1 USA revision)
│   └── pokecrystal.sym          # Symbol table generated by RGBDS rgblink (-n pokecrystal.sym)
├── data/
│   ├── knowledge_graph/         # Parsed AST from pret/pokecrystal (wild tables, trainers, moves, maps)
│   ├── states/                  # Golden save states (.state) + recurrent memory (.pt) checkpoints
│   └── schema/                  # JSON schemas for nav_memory.json and subgoal_spec.json
├── src/
│   ├── core/
│   │   ├── sym_parser.py        # Parses pokecrystal.sym -> Bank:Address tuples (zero hardcoded hex)
│   │   ├── wram_crystal.py      # GBC banked WRAM reader (Banks 0-7 via pyboy.memory[bank, addr])
│   │   ├── rtc_controller.py    # MBC3 Real-Time Clock injection (Morning/Day/Night deterministic lock)
```

```
│   │   └── pyboy_env.py            # Headless Gymnasium wrapper with double-press & warp-settle physics
│   ├── tier3_workers/
│   │   ├── worker_a_autopilot.py    # Deterministic A*, doorway sidestep, HM/Menu macro state machines
│   │   ├── worker_b_explorer.py     # 10M-param Recurrent PPO + Action-Effect Head + Visited Binary Mask
│   │   └── worker_c_combat.py       # Metamon Gen2OU Transformer wrapper + PokéChamp minimax boss search
│   ├── tier2_broker/
│   │   ├── interrupt_router.py      # 60Hz hardware tick monitor (wBattleMode, warps, HP/PP floors)
│   │   ├── saycan_gate.py           # Affordance scorer (p_LLM * V_RL) + 100x fast combat rollout engine
│   │   ├── ghost_scouts.py          # 5x async multiprocessing PyBoy scouts writing to SQLite WAL / shm
│   │   └── critique_verifier.py     # Post-condition RAM predicate evaluator & Craftax 10k stall breaker
│   ├── tier1_director/
│   │   ├── planner_agent.py         # LLM Meta-Planner (subgoal emission, veto diagnostics recovery)
│   │   ├── lstkag_retriever.py      # Deterministic query engine over pret/pokecrystal JSON tables
│   │   ├── teduo_mask.py            # Goal-conditioned state abstraction mask F(x, g) -> s_g
│   │   └── eureka_synthesizer.py    # Sandboxed AST synthesis of custom reward functions R_g(s,a,s')
│   └── training/
│       ├── vec_env_shm.py           # Shared-memory zero-copy vectorized PyBoy runner (64-128 workers)
│       ├── frontier_buffer.py       # Craftax Frontier Practice buffer (20% replay from State + Hidden)
│       └── ppo_trainer.py           # TorchCompile PPO loop with proportional reward scaling (/ 8.0)
└── tests/
    ├── states_fixtures/             # 25 golden .state fixtures (Pallet/NewBark, SproutTower, Goldenrod, IcePath)
    ├── test_phase1_substrate.py     # Determinism, sym parsing, banked WRAM, double-press physics
    ├── test_phase2_autopilot.py     # A*, ledges, off-screen NPC cache, doorway traps
    ├── test_phase3_explorer.py      # Action-effect loss convergence, anti-loop penalties, stall cap
    ├── test_phase4_combat.py        # Battle RAM translation, 100x rollout speed (<2s), Whitney test
```

└── test_phase5_handoff.py # End-to-end New Bark -> Violet Gym clear without human intervention

2. Unconsidered Engineering Factors (Critical for *Pokémon Crystal*)

Before writing code, six non-obvious hardware and systems traps in *Pokémon Crystal* (GBC / MBC3) must be engineered into the core substrate:

2.1 Zero-Hardcoded Hex via pokecrystal.sym Parsing

Unlike *Pokémon Red*, where community projects copy-paste raw hex addresses (0xD35E), *Pokémon Crystal* uses **8 switchable 4KB Work RAM banks** (WRAM0 at 0xC000–0xCFFF and switchable WRAM1–WRAM7 mapped into 0xD000–0xDFFF, controlled by hardware register SVBK at 0xFF70). Furthermore, addresses shift between Crystal v1.0 and v1.1.

- **Solution:** Clone pret/pokecrystal and compile the ROM using RGBDS (make), which outputs pokecrystal.sym. Every symbol is formatted as BB:AAAA SymbolName (e.g., 01:D4E2 wPlayerStandingMapX).
- **PyBoy Banked Direct Access:** Instead of reading pyboy.memory[0xD4E2] (which reads whichever WRAM bank happens to be active on that CPU cycle and corrupts your state vector during screen transitions), always read pyboy.memory[bank, addr] using the exact (bank, addr) tuple parsed from pokecrystal.sym.

Logical Domain	Canonical pokecrystal.sym Symbol	Bank	Function in Hybrid Architecture
Map & Spatial	wMapGroup, wMapNumber	WRAM1 (01)	Unique 2-byte compound Map ID (group, number)
Player Tile Coords	wPlayerStandingMapX, wPlayerStandingMapY	WRAM1 (01)	Player tile position (+4 offset padding in pokecrystal map headers)

Facing Direction	wPlayerDirection	WRAM1 (01)	Bits 2–3 encode DOWN=0, UP=4, LEFT=8, RIGHT=12 (governs double-press)
Off-Screen NPC Array	wMapObjects (16 * 16 bytes), wObjectStructs (13 * 40 bytes)	WRAM1 (01), WRAM0 (00)	Active sprites vs. map object templates (critical for NPC collision caching)
Battle Interrupts	wBattleMode	WRAM1 (01)	0 = Overworld, 1 = Wild Battle, 2 = Trainer Battle (60Hz interrupt trigger)
Party State	wPartyCount, wPartySpecies, wPartyMons (6 * 48 bytes)	WRAM1 (01)	Species, HP, Level, Status, Moves, PP, Held Item, Experience (3-byte big-endian)
Enemy Battle State	wEnemyMon, wEnemyMonSpecies, wEnemyMonHP, wEnemyMonMoves	WRAM1 (01)	Live opponent state fed to Worker C (Metamon) on battle interrupt
Progression Gates	wJohtoBadges, wKantoBadges, wEventFlags (250 bytes)	WRAM1 (01)	Bitmasks for SayCan prerequisite checks and TEDUO state masking

Real-Time Clock	wTimeOfDay, hHours, hMinutes	WRAM1 / HRAM	0 = Morning (04:00–09:59), 1 = Day (10:00–17:59), 2 = Night (18:00–03:59)

2.2 Deterministic MBC3 Real-Time Clock (RTC) & PRNG Seeding

Pokémon Crystal uses an MBC3 hardware Real-Time Clock. If 64 parallel RL workers run at $150\times$ real-time speed while reading the host OS system clock, wild encounter tables (data/wild/johto_grass.asm) will flip unpredictably between Morning, Day, and Night mid-rollout, breaking reproducibility.

- **Solution:** Implement RTCController in src/core/rtc_controller.py that overrides hHours and wTimeOfDay in RAM at the start of every episode (or locks RTC registers via PyBoy memory hooks) so time of day is a **controllable environment parameter** passed by Tier 1 (target_time_of_day: Literal["MORN", "DAY", "NITE"]). Similarly, lock the hardware divider and PRNG bytes (hRandomAdd, hRandomSub) during deterministic unit tests.

2.3 Concurrency-Safe Shared Navigation Graph (SQLite WAL + Shared Memory)

When 5 Ghost Scouts (Tier 2) and 1 Main Agent (Tier 3) simultaneously discover tiles and off-screen NPC positions, writing to a flat nav_memory.json file causes race conditions and corrupted JSON reads.

- **Solution:** Store the persistent world graph in an in-memory/disk **SQLite database in WAL (Write-Ahead Logging) mode** (nav_memory.db) with tables maps(group_id, map_id, width, height), tiles(group_id, map_id, x, y, walkability, tile_id, last_seen_step), npcs(group_id, map_id, obj_idx, x, y, movement_type, is_trainer), and warps(src_group, src_map, src_x, src_y, dst_group, dst_map, dst_x, dst_y). Expose a **read-only connection** (?mode=ro) to Tier 1 (planner_agent.py) so the LLM can never alter map state.

2.4 Asynchronous Non-Blocking LLM Planning & Watchdog Recovery

If Tier 1 (LLM Director) takes 4–8 seconds to reason over a sub-goal, blocking the vectorized training or scout pool wastes thousands of emulator frames.

- **Solution:** When a sub-goal finishes or stalls, the main emulator freezes on a safe overworld frame while Ghost Scouts continue mapping adjacent edges asynchronously. Every LLM output is validated against a strict **Pydantic schema** with AST sandbox verification for Eureka reward snippets (banning import, open, exec, eval, __).

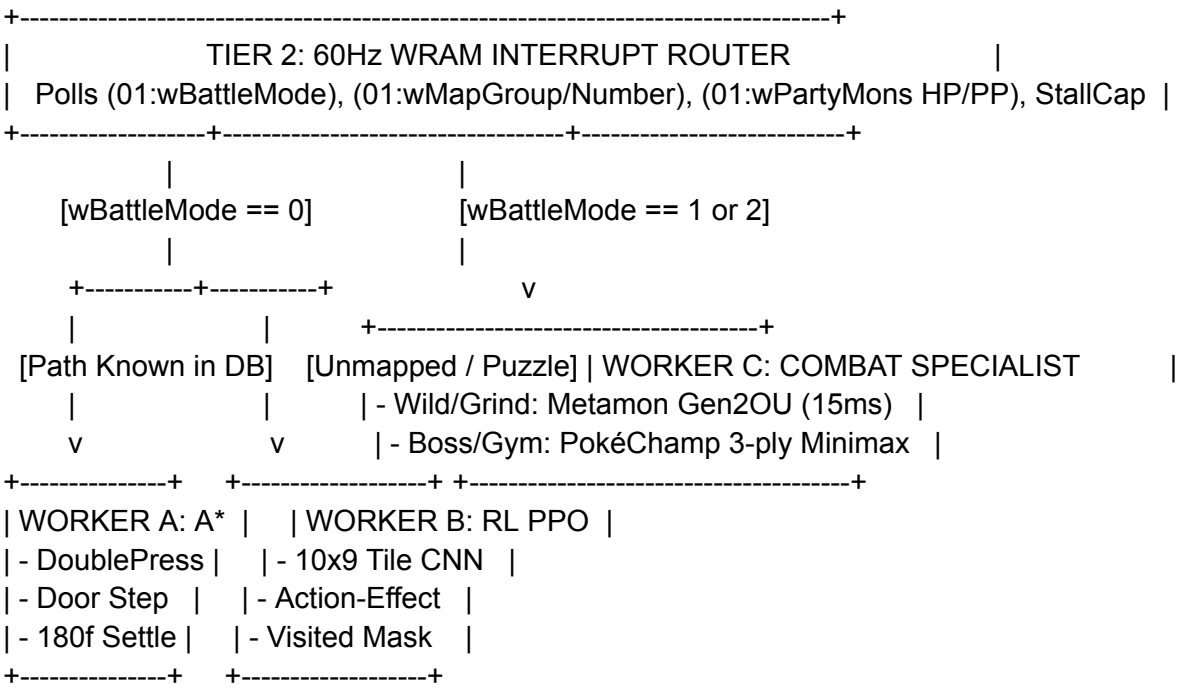
2.5 Move-Replacement & Evolution Interrupt Handler

When a Pokémon levels up in combat and learns a 5th move (e.g., Quilava learning Flame Wheel at Level 14 or Smokescreen), or attempts to evolve post-battle, the game enters a modal dialog inside or immediately after wBattleMode that breaks standard battle or overworld policies.

- **Solution:** The Tier 2 Interrupt Router monitors wMoveSelectionMenuType and evolution state flags. For move replacement, Tier 2 computes the expected DPS/utility delta of the new move vs. the 4 current moves using the LSTKAG move table; if the new move strictly dominates the weakest move (e.g., replacing Tackle with Flame Wheel or deleting a non-HM normal move), a deterministic macro overwrites that slot. (Note: in *Crystal*, HMs cannot be overwritten without the Blackthorn Move Deleter, so Tier 1 strictly assigns HMs like Cut or Whirlpool to designated utility slots).

3. Core Subsystem Technical Specifications

3.1 Tier 3 Workers (Low-Level Execution Substrate)



Worker A: Deterministic A^* & Programmatic Macro-Skills (worker_a_autopilot.py)

- **Input Contract:** AutopilotGoal(target_map: tuple[int,int], target_xy: tuple[int,int], action_macro: Optional[str])
- **Movement Physics Wrapper (step_tile(direction)):**
  - Read wPlayerDirection (01:D4DE).
  - If current_facing != direction, send 1 directional button press (4 frames hold + 4 frames release) to rotate sprite, then poll wPlayerDirection to verify rotation.
  - Send movement press (8 frames hold + 12 frames release) to step 1 tile.
  - **Doorway Exit Reflex:** Whenever wMapGroup/wMapNumber changes, hold None for 180 frames (warp settle), read the new standing tile

(wPlayerStandingMapX/Y), and if the tile immediately behind or beneath is a warp carpet (0x71 / 0x7A), execute an orthogonal sidestep before running A^*.

- **Directed A^* Cost Function:**
  - Walkable tile = 1.0
  - Wild tall grass tile (0x18) = 1.0 if subgoal.mode == "GRIND", else 4.5 (routes around grass to conserve HP/PP during traversal)
  - One-way ledge (0x20–0x27) = 2.0 if moving in the allowed jump direction, ∞ otherwise
  - Cached NPC coordinate from nav_memory.db = ∞ (if stationary) or 3×3 high-cost penalty buffer (15.0) if wandering (SPINRANDOM_SLOW).

Worker B: Goal-Conditioned Recurrent PPO Explorer (worker_b_explorer.py)
Combines PokeRL's anti-loop spatial masking with Rekursiv AI's **Craftax** architecture:

- **Action Space:** Discrete(5) — ['UP', 'DOWN', 'LEFT', 'RIGHT', 'A'] (START and SELECT are hardware-masked out; B is automatically pulsed during text-box dismiss macros).
- **Observation Space (O_t):**
  - screen_stack: uint8[4, 72, 80] — 4 stacked downsampled grayscale frames.
  - visited_mask: float32[1, 72, 80] — Centered per-map binary mask where visited (x, y) tiles on the current (wMapGroup, wMapNumber) are 1.0 and unvisited tiles are 0.0.
  - local_tile_grid: int32[9, 10] — Exact on-screen collision/tile ID matrix read from wTilemap + cached off-screen NPC markers within a 9×10 viewport.
  - teduo_state_vec: float32[32] — Masked WRAM vector filtered by Tier 1's TEDUO mask $F(x, g) \to s_g$.
- **Neural Architecture ($\approx 8.5M$ Parameters):**
  - **Visual + Mask Encoder:** 3-layer NatureCNN over concatenated [screen_stack, visited_mask] (5 x 72 x 80) $\to 256 - d$.
  - **Craftax Local Board Encoder:** Embedding layer (256 tile IDs $\to 16 - d$) + 2-layer 3×3 CNN over local_tile_grid (9 x 10) $\to 128 - d$.
  - **State & Goal Encoder:** 2-layer MLP over teduo_state_vec $\to 128 - d$.
  - **Recurrent Core:** GRUCell(input_size=512, hidden_size=512) (retains memory of off-screen NPCs and split-floor blockers across steps).
  - **Three Output Heads:**
    1. **Policy Head $\pi_\theta(a_t \mid h_t)$:** Linear(512, 5)
    2. **Value Head $V_\phi(h_t)$:** Linear(512, 1) (also queried by Tier 2 for SayCan affordance gating!)
    3. **Craftax Auxiliary Action-Effect Head $P_\psi(\Delta \text{screen} \mid h_t, a_t)$:** Linear(512 + 5, 1) with binary cross-entropy loss predicting whether action a_t alters the center 72×80 frame by more than 2% pixel L1 norm (teaches immediate wall, ledge, and NPC affordances).
- **Reward Scaling & Anti-Loop Shaping (Rekursiv Craftax + PokeRL):**

- **No $[-1, 1]$ Reward Clipping:** Raw step rewards are divided by a fixed constant REWARD_SCALE = 8.0 so sub-goal completion (+10.0 / 8.0 = +1.25) always dominates novel tile discovery (+0.05 / 8.0 = +0.00625).
- **3-Layer Anti-Loop Penalty:**

- $R_{loop}(t) = -\frac{1}{8.0}\Big(0.01 \cdot I[visits(x,y) > 3] + 0.03 \cdot I[visits(x,y) > 5] + 0.05 \cdot I[cycle_{20}(a_{t-20:t}$

Worker C: Combat Specialist & Fast Rollout Simulator (worker_c_combat.py)

- **State Translator (wram_to_gen2_state()):** Reads wPartyMons and wEnemyMon from Bank 01 on every battle turn prompt (wBattleMenuCursorPosition ready) and constructs a canonical Gen 2 battle state tensor compatible with UT-Austin-RPL/metamon (Superkazam / Kakuna Gen2OU checkpoints).
- **Dual-Mode Execution:**
  - **Wild / Routine Trainer Mode:** Single forward pass (<15ms) through the Metamon Transformer conditioned on a 1-hot mission scalar: FIGHT_OPTIMAL (trainer battle), GRIND_FAST_OHKO (select highest-PP 100%-acc move that KOs), CATCH_TARGET (status + Pokéball if wEnemyMonSpecies == target_id, else Run), or EVADE_LOW_HP (immediate Run).
  - **Boss / Gym Leader Mode (wBattleMode == 2 & Boss ID):** Runs a **3-ply PokéChamp Expectiminimax Search** using a lightweight Cython/Rust Gen 2 damage & speed calculator (calc_gen2_damage(attacker, move, defender, weather, screens)) where leaf nodes are scored by the Metamon value head $V_{combat}(s)$.

3.2 Tier 2: Affordance Handoff Broker (src/tier2_broker/)

1. **SayCan Affordance Gating (saycan_gate.py):**
   Before dispatching any Tier 1 candidate sub-goal $g_k \in \{g_1, \dots, g_K\}$ with LLM log-probability score $p_{\text{LLM}}(g_k \mid s)$, Tier 2 computes the physical affordance $p_{\text{affordance}}(g_k \mid s)$:
   - **For Navigation/Puzzle Sub-goals:** Queries a deterministic prerequisite checklist (e.g., has_badge(PLAINBADGE) and party_has_move(STRENGTH)) multiplied by the normalized value head $\sigma(V_\phi(h_t, g_k))$ of Worker B.
   - **For Boss Sub-goals (CHALLENGE_GYM):** Clones the current party state from WRAM and runs **100 vectorized headless combat rollouts** in Python/Cython against the boss's canonical roster from data/knowledge_graph/trainers.json.
   - **Veto Rule:** If $p_{\text{affordance}}(g_k \mid s) < \tau_{\text{ready}}$ (default 0.65 for bosses, 0.20 for navigation), Tier 2 **vetoes** the sub-goal without moving a single step in the main emulator and returns a structured VetoDiagnosticReport JSON to Tier 1.
2. **5x Parallel Ghost Scouts (ghost_scouts.py):**
   Whenever the agent enters a map where unexplored_border_tiles > 0 or A^* returns

NoPathFound, Tier 2 forks the current PyBoy state (pyboy.save_state(io_bytes)) across **5 background worker processes**:

- Scout 1: Wall-follower (Left-hand rule) with walk_wait=16
- Scout 2: Wall-follower (Right-hand rule) with walk_wait=16
- Scout 3: North-East biased random walk with orthogonal sidestep reflex (walk_wait=20)
- Scout 4: South-West biased random walk with orthogonal sidestep reflex (walk_wait=20)
- Scout 5: Worker B policy at high temperature ($\tau = 1.8$)

Every discovered walkable tile, one-way ledge jump, warp transition, and off-screen NPC coordinate (wMapObjects) is written immediately to nav_memory.db. The millisecond a scout touches the sub-goal target coordinate or warp, Tier 2 halts Worker B and switches the main emulator to **Worker A (A^* Autopilot)** along the newly discovered path.

3. **Craftax Hard Stall Cap (critique_verifier.py):**
   Tracks steps_since_last_milestone_or_subgoal. If steps_since_last_subgoal >= 2,500 (local sub-goal stall) or steps_since_last_milestone >= 10,000 (Craftax hard cap), Tier 2 halts execution, rolls back to the pre-subgoal checkpoint if trapped in a soft-lock (e.g., mis-pushed boulder in Ice Path), and escalates to Tier 1 with the full trajectory heatmap.

3.3 Tier 1: LLM Meta-Planner & Synthesis Engine (src/tier1_director/)

Tier 1 is called asynchronously via a structured tool-use protocol (SubGoalSpec).

JSON

```
{
  "subgoal_id": "goldenrod_counter_whitney_01",
  "worker_mode": "AUTOPILOT_THEN_GRIND",
  "target_map": [11, 4],
  "target_coords": [8, 12],
  "rtc_override": "DAY",
  "combat_directive": {
    "mode": "CATCH_TARGET",
    "target_species_sym": "DROWZEE",
    "hp_retreat_threshold": 0.30
  },
  "teduo_active_wram_keys": [
    "wMapGroup", "wMapNumber", "wPlayerStandingMapX", "wPlayerStandingMapY",
    "wPartyCount", "wPartySpecies", "wItemCount"
  ],
  "termination_predicate_expr": "PARTY_HAS_SPECIES('DROWZEE')",
  "max_step_budget": 1500
}
```

- **LSTKAG Disassembly Knowledge Graph (lstkag_retriever.py):**
  During repo build (make data), a Python parser compiles pret/pokecrystal assembly files (data/wild/*.asm, data/trainers/parties.asm, data/pokemon/base_stats/*.asm, data/Pokemon/evos_attacks.asm, maps/*.asm) into deterministic SQLite/JSON tables. Tier 1 queries find_counter_pokemon(boss_id="WHITNEY", max_map_distance=2, time_of_day="DAY") to get exact encounter rates, base stats, and in-game trade NPCs without hallucinating Gen 2 mechanics.
- **TEDUO State Abstraction Mask (teduo_mask.py):**
  Maps teduo_active_wram_keys into a binary mask $M_g \in \{0,1\}^{160}$ over the full 160-feature WRAM vector, zeroing out ~80% of irrelevant features before feeding teduo_state_vec to Worker B.

4. Mass Simulation & Training Optimization Architecture

To train Worker B (the 8.5M-parameter Recurrent PPO Explorer) and run mass campaign evaluations at **40,000+ emulator frames per second (FPS)** on a single workstation or GPU node, implement the following four systems optimizations:

4.1 Zero-Copy SharedMemory Vectorized PyBoy Pool (vec_env_shm.py)

Standard gymnasium.vector.AsyncVectorEnv pickles numpy arrays across Python multiprocessing pipes on every step, spending >60% of CPU time on IPC serialization.

1. Pre-allocate a single contiguous POSIX shared memory block (multiprocessing.shared_memory.SharedMemory) at startup for $N = 64$ or 128 headless PyBoy processes (window="null", sound=False, sound_emulated=False).
2. Each PyBoy worker writes its 72×80 downsampled screen, 72×80 visited mask, 9×10 local tile grid, and $32 - d$ WRAM vector **directly into its slice of the shared memory buffer**.
3. On the GPU host process, wrap the shared memory buffer in torch.frombuffer(...).pin_memory() and transfer the entire batch of 128 environments to CUDA in a single asynchronous DMA transfer (non_blocking=True).
4. **Frame-Skip & Action Decimation:** Each RL step executes a full macro-tick (16–20 emulator frames for turn+step) inside Cython/PyBoy before returning to Python, yielding an effective $16 \times$ speedup over frame-level RL.

4.2 Rekursiv Craftax "Frontier Practice" Curriculum Buffer (frontier_buffer.py)

To prevent parallel RL workers from wasting 90% of their steps in New Bark Town or early routes:

1. Maintain a priority ring buffer of **Frontier Checkpoints**. Whenever any worker reaches a new (wMapGroup, wMapNumber) floor, solves a new wEventFlags bit, or achieves a

new maximum campaign score for the first time, serialize a tuple:

2. $FrontierEntry = \left(PyBoy`.state`bytes, h_t \in R^{512} GRUhiddenstate, SQLite`nav_memory`snapshot\right)$
3. **The 20% Frontier Reset Rule:** On every episode reset (done or 10,000-step stall cap), **80% of workers** reset to the current curriculum stage start state, while **20% of workers** sample a state directly from the FrontierBuffer—restoring **both** the PyBoy RAM state *and* the agent's recurrent hidden state h_t. This concentrates high-density rollout practice directly on mid/late-game bottlenecks (Sprout Tower 3F, Union Cave B2F, Morty's invisible floor puzzle, Ice Path boulder rooms).

4.3 Multi-Stage Parallel Save-State Bank (tests/states_fixtures/)

Do not train or test sequentially from the title screen. Build a bank of **25 canonical .state fixtures** spanning the entire Johto campaign so all 25 bottlenecks can be trained and regression-tested in parallel from Day 1:

- 01_elm_lab_starter.state, 02_route29_ledgedrop.state, 03_sprout_tower_splitfloor.state, 04_falkner_gym.state, 05_union_cave_b1f.state, 06_azalea_slowpoke_well.state, 07_bugsy_gym_spinarak_web.state, 08_ilex_forest_farfetchd.state, 09_goldenrod_underground.state, 10_whitney_pre_gym.state, 11_burned_tower_b1f.state, 12_ecruteak_morty_invisible_floor.state, 13_olivine_lighthouse.state, 14_cianwood_chuck_boulder.state, 15_mahogany_rocket_hq.state, 16_pryce_ice_gym_slider.state, 17_goldenrod_radio_tower.state, 18_ice_path_b1f_boulders.state, 19_blackthorn_clair_gym.state, 20_victory_road_crystal.state, 21_indigos_elite_four.state.

5. Phase-Gated Agentic Implementation Plan (Spec-Driven Development)

Feed the following 6 phases sequentially to your agentic coding tool. **Rule for the Coding Agent:** *Do not advance to Phase $N + 1$ until the verification command for Phase N exits with code 0 and meets all quantitative performance gates.*

[Phase 1: Substrate & .sym] ---> [Phase 2: Worker A + Scouts] ---> [Phase 3: Worker B (Craftax PPO)]
|
[Phase 6: Full Campaign CI] <--- [Phase 5: Tier 1 + Eureka] <--- [Phase 4: Worker C + SayCan Gate]

Phase 1: Deterministic Emulation Substrate & Banked .sym Reader

- **Deliverables:**
  - Makefile rule that clones pret/pokecrystal, compiles roms/pokecrystal.gbc and roms/pokecrystal.sym, and verifies SHA1 checksum.

    - src/core/sym_parser.py and src/core/wram_crystal.py implementing banked memory reads (pyboy.memory[bank, addr]) for all table symbols without a single hardcoded hex address.
    - src/core/rtc_controller.py and src/core/pyboy_env.py implementing double-press movement, 180-frame warp settle, and SharedMemory vectorization (src/training/vec_env_shm.py).
- **Agentic Verification Gate (pytest tests/test_phase1_substrate.py):**
    - **Test 1 (Determinism):** Run two headless instances from 01_elm_lab_starter.state with fixed PRNG and RTC seed executing 1,000 random macro-steps; assert byte-for-byte identical WRAM Bank 1 state at step 1,000.
    - **Test 2 (Bank Safety):** Trigger a screen transition warp while polling wPartyMons and wMapGroup; assert zero corrupted/zeroed reads across all 180 transition frames.
    - **Test 3 (Throughput Benchmark):** Run 64 vectorized SharedMemory workers for 10,000 macro-steps; assert aggregate throughput $\geq 25,000 FPS$ on CPU.

Phase 2: Worker A (A^* Autopilot), Off-Screen NPC Cache & 5x Ghost Scouts

- **Deliverables:**
    - src/tier2_broker/ghost_scouts.py + SQLite WAL nav_memory.db schema.
    - src/tier3_workers/worker_a_autopilot.py with directed A^* (supporting one-way ledges, grass cost weighting, doorway exit orthogonal sidestep, and cached off-screen NPC avoidance from wMapObjects).
- **Agentic Verification Gate (pytest tests/test_phase2_autopilot.py):**
    - **Test 1 (Doorway Exit Trap):** Load a state inside Elm's Lab and New Bark player house; execute Exit_Building -> Re_Enter -> Exit_Building 20 consecutive times with **100% success** and zero re-entry loops.
    - **Test 2 (Off-Screen NPC Persistence):** Load Violet City or Route 30; spot an NPC on screen, walk 12 tiles away so the NPC leaves RAM visibility, then command A^* to path across the NPC's tile; assert A^* routes cleanly around the cached NPC coordinate.
    - **Test 3 (Ghost Scout Discovery):** Load 03_sprout_tower_splitfloor.state with an empty nav_memory.db; launch the 5 Ghost Scouts and assert the exit ladder path is written to nav_memory.db and executed by Worker A in $< 15 seconds$ **wall-clock time**.

Phase 3: Worker B (Craftax Recurrent PPO Explorer + Frontier Buffer)

- **Deliverables:**
    - src/tier3_workers/worker_b_explorer.py implementing the $8.5M$-param NatureCNN + 9×10 Local Board CNN + GRU + Action-Effect Head + Visited Binary Mask.

    - src/training/frontier_buffer.py (20% Frontier Practice replay) and src/training/ppo_trainer.py (proportional reward scaling / 8.0 and 3-layer anti-loop penalty).
- **Agentic Verification Gate (pytest tests/test_phase3_explorer.py):**
    - **Test 1 (Action-Effect Convergence):** Train the Action-Effect head on 50,000 steps of random/policy rollouts in 05_union_cave_b1f.state; assert wall/ledge/NPC bump prediction accuracy reaches **$\ge 95\%$** and wall-bump action rate drops below **3%**.
    - **Test 2 (Zero-Loop Guarantee):** Evaluate Worker B across 100 episodes in 03_sprout_tower_splitfloor.state and 05_union_cave_b1f.state; assert **0 episodes** get stuck in a 20-step action loop or menu spam.

Phase 4: Worker C (Metamon Combat + Minimax) & SayCan Readiness Gate

- **Deliverables:**
    - src/tier3_workers/worker_c_combat.py integrating the wram_to_gen2_state() bridge, Metamon Gen2OU inference, and 3-ply PokéChamp minimax boss search.
    - src/tier2_broker/interrupt_router.py (60Hz wBattleMode hardware switch) and src/tier2_broker/saycan_gate.py (100x headless combat rollout engine).
- **Agentic Verification Gate (pytest tests/test_phase4_combat.py):**
    - **Test 1 (Rollout Latency):** Execute 100 full simulated battles between a Level 18 party and Whitney's roster (Clefairy Lv 18, Miltank Lv 20); assert 100 rollouts complete in $< 1.5 seconds$.
    - **Test 2 (SayCan Veto & Counter Validation):** Verify that saycan_gate.evaluate("CHALLENGE_WHITNEY") returns VETO (win_rate < 0.20) for a solo Male Level 18 Quilava, and returns APPROVE (win_rate > 0.80) when a Level 18 Female Machop or Geodude is added to the party state.
    - **Test 3 (Live PyBoy Boss Clear):** Load 10_whitney_pre_gym.state with the approved counter team and run Worker C inside PyBoy across 20 seeded trials; assert **$\ge 85\%$ live win rate** against Whitney.

Phase 5: Tier 1 LLM Director, LSTKAG Graph & Sandboxed Eureka

- **Deliverables:**
    - data/knowledge_graph/ builder script parsing pret/pokecrystal into SQLite/JSON.
    - src/tier1_director/lstkag_retriever.py, teduo_mask.py, eureka_synthesizer.py, and planner_agent.py.
- **Agentic Verification Gate (pytest tests/test_phase5_handoff.py):**
    - **Test 1 (Zero-Hallucination Counter Retrieval):** Pass the Whitney SayCan Veto report to planner_agent.py; assert Tier 1 queries lstkag_retriever and emits the valid sub-goal chain Catch(DROWZEE, Route_34) -> Trade(MACHOP, Goldenrod_Dept_5F) or Catch(GEODUDE, Route_46).

- **Test 2 (Eureka Boulder Puzzle Synthesis):** Present 18_ice_path_b1f_boulders.state to eureka_synthesizer.py; assert the generated reward function passes AST security validation, executes in <0.1ms/step, and guides Worker B/Scouts to drop the boulder into the target hole (wMapObjects coordinate match).

Phase 6: End-to-End Campaign Regression & Telemetry CI

- **Deliverables:**
  - Full closed-loop integration script (scripts/run_campaign.py) with automated telemetry dashboard tracking: **Badges Earned**, **Total Emulator Frames**, **Total LLM Tokens Consumed**, **Worker Split % (A^* vs. RL vs. Combat)**, and **SayCan Vetoes Triggered**.
- **Final Benchmark Acceptance Gate:**
  - **Slice 1 (Early-Game Vertical Slice):** Autonomous clear from 01_elm_lab_starter.state through **Falkner (Badge 1)**, **Sprout Tower**, **Union Cave**, **Bugsy (Badge 2)**, **Ilex Forest (Cut gate)**, and **Whitney (Badge 3)** in $< 15M$ **total LLM tokens** ($\ge 99\%$ token reduction vs. pure LLM harnesses).
  - **Slice 2 (Full Johto 8-Badge + Red Campaign):** Parallel validation across all 25 golden .state fixtures followed by a continuous end-to-end run through all 8 Johto Badges, Elite Four Champion Lance, and Mt. Silver Red.

6. Copy-Paste CLAUDE.md Root Prompt for Your Coding Agent

Drop this file into the root of your repository before initializing your agentic coding CLI:

Markdown
```
# CLAUDE.md — Project CRYSTAL-HYBRID Operating Directives

## 1. Architectural Law
You are building a 3-Tier Hybrid RL-LLM system for Pokémon Crystal (GBC) on PyBoy.
- NEVER hardcode hex memory addresses. Always resolve symbols dynamically from `roms/pokecrystal.sym` via `src/core/sym_parser.py` and read banked memory via `pyboy.memory[bank, addr]`.
- NEVER allow Tier 1 (LLM) to emit raw button presses (`Up`, `Down`, `A`, `B`) or gain write access to `nav_memory.db`.
- NEVER allow Worker B (Overworld RL) to press `START` or `SELECT`, or clip rewards to `[-1, 1]`. Always scale rewards by `/ 8.0`.
- ALWAYS route combat at 60Hz via hardware interrupt when `01:wBattleMode != 0` directly to Worker C (zero LLM calls during routine wild battles).

## 2. Phase-Gated Workflow
Work strictly one phase at a time (Phase 1 -> Phase 6).
1. Read the current phase specification and write failing tests in `tests/test_phaseX_*.py` first.
```

2. Implement the module with strict Python 3.11+ type annotations and Pydantic schemas.
3. Run `pytest tests/test_phaseX_*.py -v`. Do not proceed to the next phase until all assertions and FPS/win-rate benchmarks pass.